Advanced Retrieval Augmented Generation (RAG) system for Multi-Store Retrieval

Tan Heng Joo
Singapore Institute of Technology
Singapore, 138683
2201497@sit.singaporetech.edu.sg

***Abstract*—Large Language Models (LLMs) excel in generating responses for multiple tasks, however, they are constrained by static, pre-trained knowledge, leading to factual inaccuracies and an inability to reason over real-time information. Retrieval-Augmented Generation (RAG) emerged as a foundational solution to ground LLMs in external data providing real-time information with contextual understanding. Early RAG frameworks typically rely on a single vector store for unstructured text retrieval, as it is able to return highly similar text to the query. However, this one "one-size-fits-all" approach struggles when dealing with real-world data as it comes in various formats such as unstructured documents, structured tabular data, knowledge graphs and semi-structured formats like JSON and XML files. These files are typically stored in various datastores, including vector datastores, SQL datastores and graph datastores. Recent advances in embedding models and graph retrieval frameworks have made multi-store architectures a feasible idea for handling real-world data complexity. This literature review synthesises insights from research papers to trace the evolution of the RAG model from the initial Naive RAG framework to the modern Agentic RAG before proposing an architecture that utilises Agentic RAG to orchestrate multi-store retrieval across various datastore infrastructures.**

***Index Terms*—Retrieval-Augmented Generation, RAG, Naive RAG, Advanced RAG, Modular RAG, Graph RAG, Agentic RAG, Multi-Store Retrieval**

I. Introduction

Large Language Models (LLM) such as OpenAI's GPT-4o, Google's Gemini-2.5 Pro and xAI's Grok-3 have demonstrated outstanding capabilities in understanding and generating human-like text as they are trained on enormous datasets that are publicly available on the internet [1]. This extensive training allows LLMs to be able to access a broad range of knowledge across diverse domains and understand the concepts from these fields simultaneously to answer the user's question.

However, their knowledge is fundamentally limited by the static data they were trained on, which results in producing inaccurate information with confidence when asked on domains that are not well represented or not present in their training dataset. Additionally, LLMs often struggle with questions requiring real-time information or highly specialised domain knowledge due to the "knowledge cut-off date" from static training.

To address these limitations, Retrieval-Augmented Generation (RAG) was introduced as a solution by connecting the LLMs to curated and trusted, up-to-date knowledge sources to enhance their capabilities to give up-to-date information [2]. Instead of relying solely on information from data it has been trained on (parametric memory), a RAG system first retrieves relevant information from a curated knowledge database (non-parametric memory) and uses that information to generate up-to-date responses. This enhances the prompt to the LLM responsible for generating the response, by providing up-to-date, tailored and factually grounded context in the user's specific circumstance. While it sounds all encompassing in theory, in reality, not all data can be vectorised and stored in a vector datastore [4]. In today's world, there are various data types besides unstructured data such as graph and structured data which can capture connections or relationships between data.

Organizations typically manage unstructured documents (PDFs, Word Documents), structured tabular data (CSV files, Excel spreadsheets) and relational databases (transaction logs, customer records), each of this format requires a specific way to be stored and retrieved. Hence, a "one-size-fits-all" approach of a single-store RAG system is significantly limited when handling this data diversity. Faced with these limitations, multi-store retrieval architectures come to mind as it can leverage the strengths of multiple types of datastores, such as graph, structured and unstructured, to tackle the various types of data to provide accurate and contextually relevant responses to the user.

II. Traditional LLMs and Their Limitations

Large Language Models have demonstrated remarkable capabilities in natural language understanding and generation, achieving human-level performance across numerous tasks. However, their architecture and training methodology introduce several fundamental limitations that constrain their real-world applicability.

A. Knowledge Cut-off and Static Training Data

LLMs are trained on enormous datasets that often cover a broad range of knowledge across various domains, which allows them to absorb and utilise concepts from multiple fields at the same time. However, these datasets are static, resulting in the inability to provide information about events or developments that occurred after their training was completed.

B. Hallucination and Factual Inaccuracy

As the LLM relies solely on static patterns captured during its training, it may produce factually incorrect or fabricated content, which is also known as hallucination. When a prompt falls outside the scope of its parametric memory, the model attempts to fill gaps in its knowledge by extracting information that is partially correlated to the prompt, resulting in responses that contain incorrect or fabricated information [3].

C. Lack of Domain-Specific Knowledge

Similar to the previous section on hallucination, LLMs cannot reference unseen content, such as information that is unique to an organisation or field. As such, it relies on its general-purpose embeddings and statistical patterns to make imprecise guesses, often producing inaccurate explanations and fabricating details.

III. Overview of RAG

Retrieval-Augmented Generation emerged as a solution to the limitations of traditional LLMs by enhancing the generative capabilities of language models with up-to-date and precise information retrieval techniques.

A. RAG Components

RAG systems consist of three main components that work together in sequence to enhance LLM outputs with external knowledge.

- **Retrieval:** The retrieval component is responsible for identifying and extracting information from external knowledge sources to support the generation process.
- **Augmentation:** The augmentation component serves as a bridge to connect the retrieved information and the generation model by integrating the retrieved information with the original query to create prompts that are rich in context for better response generation.
- **Generation:** The generation component is where the LLM processes the augmented prompt to produce relevant responses

IV. Evolution of RAG

A. The Foundation: Naive RAG

The RAG framework has evolved significantly since it was formally introduced by Lewis et al. (2020) as a general-purpose fine-tuning approach. This initial concept, which is now often referred to as Naive RAG, combines the parametric memory of pre-trained models with a non-parametric external knowledge database which follows a linear pipeline of indexing, retrieval and generation (Figure 1). The system would take a user query, retrieve the most relevant text chunks from the data store and feed these chunks alongside the original query into the LLM to produce a response. Although the Naive RAG framework is able to produce good results on knowledge-intensive tasks, it faces significant limitations as well in areas such as retrieval precision, context integration and generating responses that are not fully supported by the retrieved text.

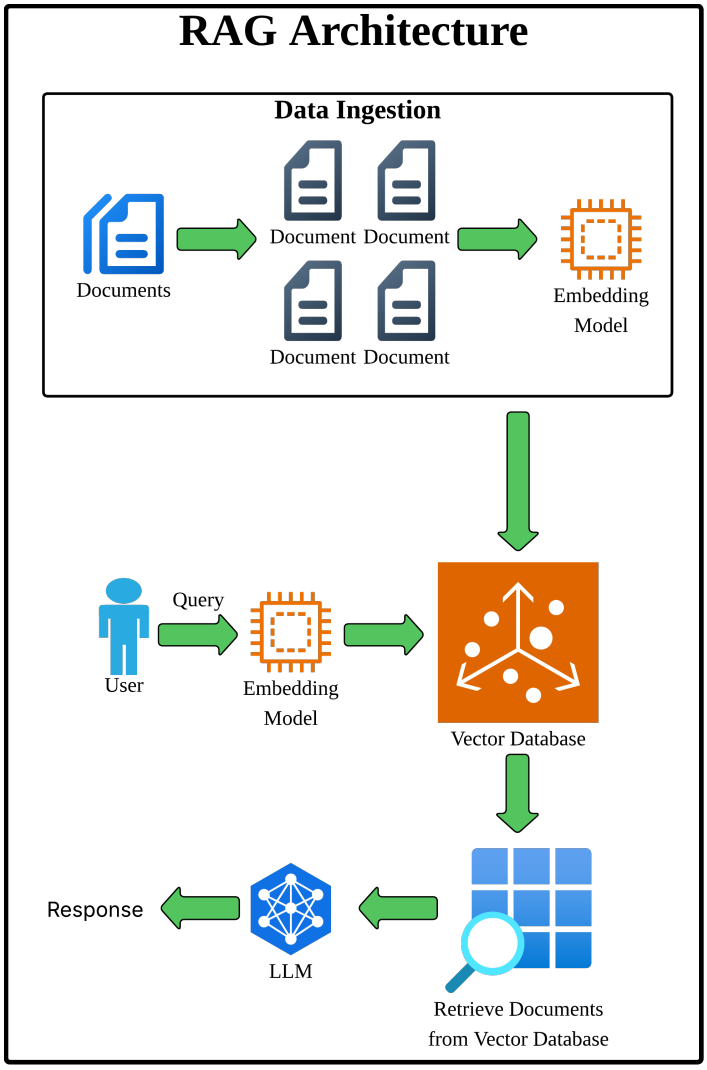

Fig. 1: Overview of Naive RAG Pipeline - Consisting of data ingestion into a vector database, user query embedding, vector similarity search to retrieve documents and LLM generation for user response.

B. Advanced RAG

To overcome the limitations proposed by the Naive RAG approach, the field evolved towards Advanced RAG, which focuses on optimising the retrieval stage of the linear pipeline by introducing pre-retrieval and post-retrieval techniques to enhance the quality and relevance of the context provided to the LLM for generation. A prime example of such pre-retrieval optimisation is the use of query rewriting and transformation, which reformulates a user's query into a clearer, more specific and better-aligned query with the knowledge base before the retrieval step, which increases the chance of finding the most relevant information from the data store [4].

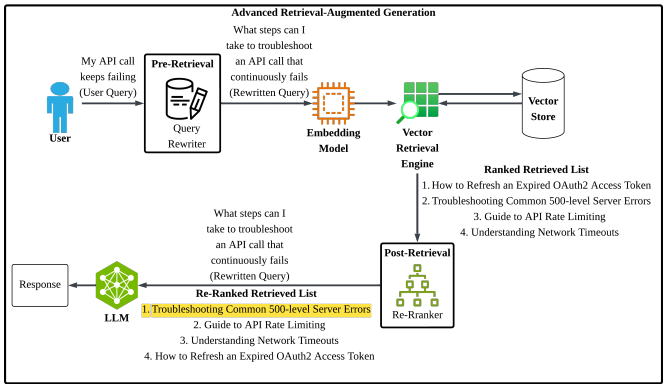

Fig. 2: Overview of Advanced RAG Pipeline - An example of an advanced RAG pipeline implementing pre-retrieval (query rewriting) and post-retrieval (re-ranking) techniques.

Example:
User Query: *"How do I make this work when my API call keeps failing?"*
Rewritten Query: *"What steps can I take to troubleshoot and resolve failing API calls, including checking authentication headers, handling rate limits, managing network timeouts, and debugging server errors?"*

The initial retrieval stage is often based on a fast similarity search, which prioritises recall, e.g which of these documents are relevant, instead of precision, e.g how relevant are these documents. Hence, there is a need for post-retrieval, and a common technique used in post-retrieval optimisation is re-ranking, which solves the issue where the initial retrieval returns documents that are semantically similar but not contextually relevant. Re-ranking solves this by after receiving a broad initial retrieval to include as many semantically similar documents as possible to maximise the chances of including all relevant information, it then utilises a cross-encoder to process the query and each document together to have a deep analysis of their semantic relevance. Finally, the model re-ranks the documents to ensure that only the most relevant documents are sent to the LLM to generate a response [2].

Example:
Query: "Troubleshoot API call failures: authentication headers, rate limits, network timeouts, server errors"
Initial Rank List:

1) Troubleshooting Common 500-level Server Errors
2) Guide to API Rate Limiting
3) Understanding Network Timeouts
4) How to Refresh an Expired OAuth2 Access Token

After Post-Retrieval Rank List:

1) **How to Refresh an Expired OAuth2 Access Token**
2) Troubleshooting Common 500-level Server Errors
3) Guide to API Rate Limiting
4) Understanding Network Timeouts

C. Modular RAG

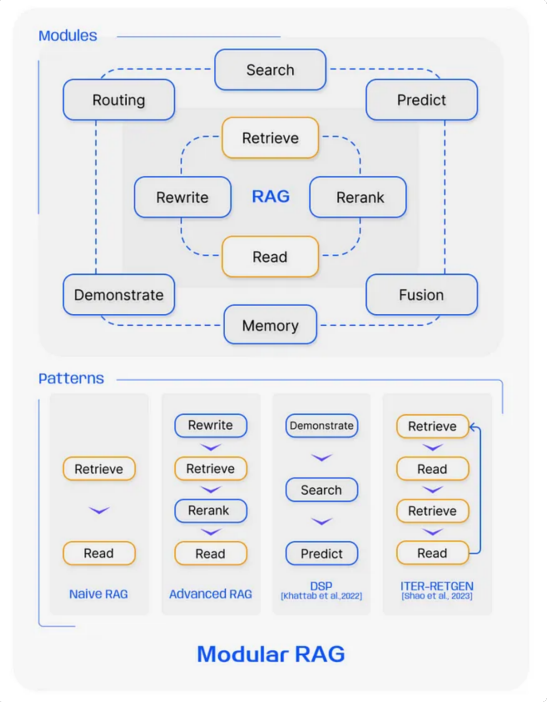

Fig. 3: Overview of Modular RAG Pipeline - Dictating reusable components that can be combined into patterns (Naive RAG, Advanced RAG, etc) allowing flexibility in architecture design.

While Advanced RAG focuses on optimising the retrieval stage within a linear pipeline, Modular RAG represents a shift in architecture towards flexibility. This model redesigns the RAG framework into a system of interchangeable modules to be more dynamic and adaptable to various workflows.

Modular RAG introduces new components such as a Search module that can query diverse sources like vector databases, search engines and knowledge graphs [5]. The Memory module is able to draw from its memory of past experiences, which will continuously improve its searches to find more relevant and accurate information as time passes. Overall, this architecture is akin to a "plug-and-play" system that can support various patterns that are tailored for specific complex tasks.

D. Graph RAG

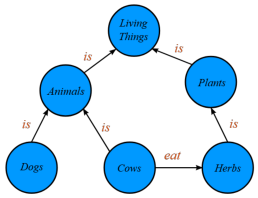

Fig. 4: Knowledge Graph - Entities modelled as nodes connected by edges to represent relationships between two nodes.

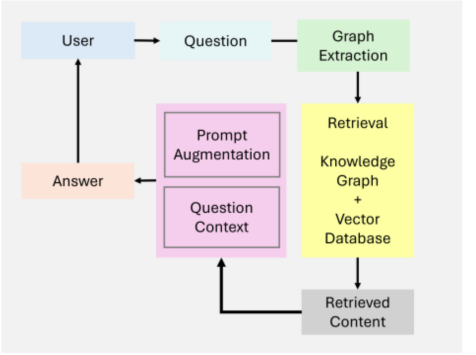

Fig. 5: Graph RAG Overview - Example of how a user's query is used to retrieve complex relationships in a graph RAG pipeline

A specialised approach that enhances retrieval by sourcing information from graph-structured data such as knowledge graphs (Figure 4) instead of unstructured text [5]. Traditional RAG struggles with queries that require multiple levels of reasoning and understanding complex relationships, which is overcome by Graph RAG as it traverse a graph of entities and their connections to provide richer and more contextual information than isolated text chunks. Resulting in not only comprehending a piece of information but also how it can relate to other pieces of information.

E. Agentic RAG

Agentic RAG is a model that embeds autonomous AI agents directly into the RAG pipeline, which acts as decision-making entities that are capable of planning, using tools, reflecting on their actions and collaborating with other agents. This was modelled after Chain-of-Thought (CoT) [3] prompting, where it was discovered that LLMs could solve complex problems if they were prompted to think step by step.

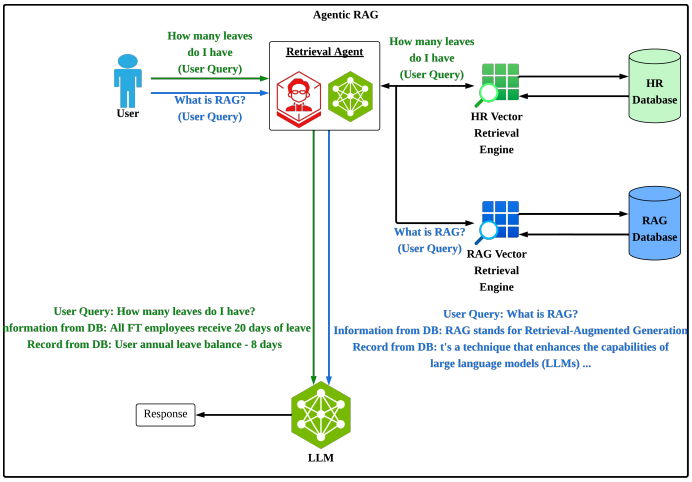

Fig. 6: Agentic RAG Overview - An example of an Agentic RAG pipeline where the agent serves as a retriever router to select which datastore to conduct retrieval from based on the user's query.

Introducing a new level of being dynamic, the agents allow the system to take ownership of its reasoning process by being able to determine the best steps to take to tackle a problem. As illustrated in (Figure 6), these agents are able to act as an intelligent router that analyses incoming queries and selects the most appropriate data source to handle them. In this example, the central "Retrieval Agent" selects the appropriate data source based on the context of the query.

V. Considerations for Multi-Store Retrieval

The evolution of RAG from simple pipelines to autonomous agentic systems provides a clear solution for addressing complex, real-world information retrieval challenges. Based on the insights gained from conducting an extensive literature review and evaluation of existing RAG architectures, I arrived at my proposed architecture, an Agentic Multi-Store RAG (AMS-RAG) pipeline that combines the strengths of a vector, graph and SQL datastore retrieval. Additionally, the proposed design includes a router agent that is able to dynamically classify each rewritten query and select the most suitable datastore to conduct retrieval from, followed by a re-ranking step to collate the results before arriving at the LLM for response generation.

A critical yet under explored aspect of a multi-store RAG pipeline is the authentication and security framework in place by enforcing fine-grained access control to protect sensitive data across the various datastores. Organisations that handle sensitive data often falls under strict regulatory frameworks and industry-specific compliance standards, which creates a challenge of maintaining consistent security policies across the datastores and demand that users only access datastores for which they are authorised while preserving the performance and benefits of a multi-store architecture.

The proposed AMS-RAG pipeline will address these security challenges utilising an database authentication framework that provides:

1) **Layered Access Control:** Role-based access control control (RBAC) will be implemented with attribute-based access control (ABAC) with the former as the first layer of authentication based on the job title and the latter as the second layer of authentication based on the type of resource being retrieved and the environment of the user. These mechanism enforce least-privilege principles across the datastores.
2) **Dynamic Authorisation:** Real-time permission evaluation that considers the context of the user query and data sensitivity when routing retrieval requests.
3) **Audit Trail:** Logging and monitoring features to track data access across all the datastores

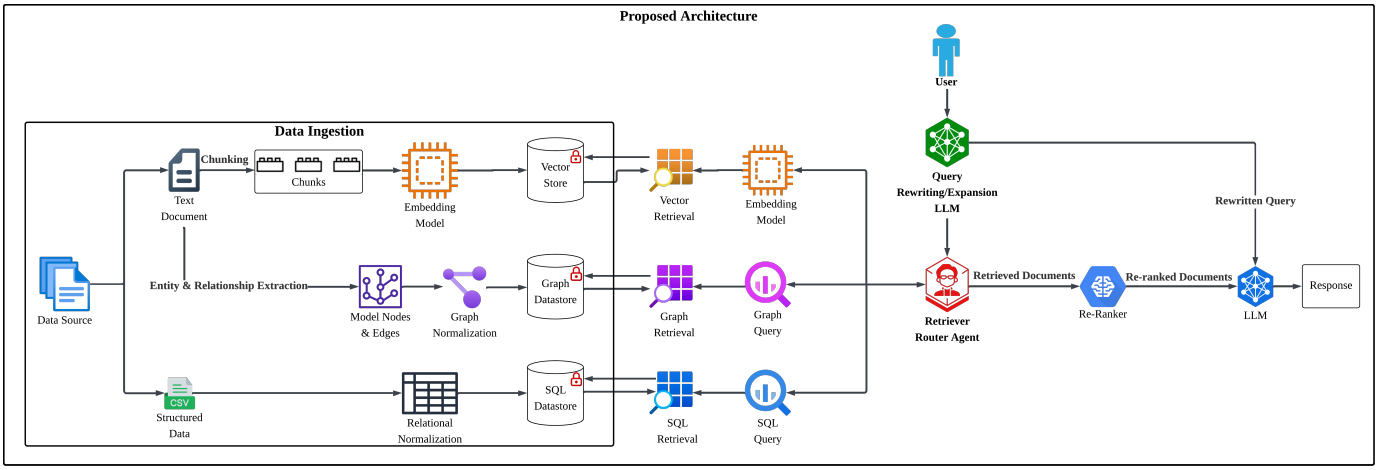

Fig. 7: Proposed Architecture

As illustrated in (Figure 7), the proposed pipeline will adopt a modular router agent framework that is broken down into two main parts:

A. Data Ingestion

In modern RAG architectures, relying on a single datastore forces a one-size-fits-all compromise. For example, semantic search engines are proficient in finding relevant text but falls short when modelling complex relationships. Graph databases reveal complex relationships through multi-hop traversal, but lack full-text similarity support. Relational databases display fast exact-match queries but are unable to handle unstructured content. Hence, by ingesting data into three different datastores, each retrieval subsystem are able to operate on inputs that are transformed optimally for them, which maximises retrieval accuracy and efficiency across the various types of queries.

1. **Vector Store:**
   When data has been extracted, it is chunked into smaller, coherent segments. Each chunk then undergoes the embedding process, where it is tokenised and encoded into fixed-length embedding vectors.
2. **Graph Datastore**
   For data to be ingested into the graph datastore it has to first undergo transformation. This is done by first extracting the entity relationship from the dataset, which will make up the nodes (entities) and edges (relations) of the graph datastore. Each node and edge will then be mapped according to the graph schema, and duplicated entities will be merged into single nodes before forming a graph datastore.
3. **SQL Datastore**
   Similar to a graph datastore, data must undergo transformation to be able to be ingested into a SQL datastore. Data is first decomposed into normalised schemas to eliminate anomalies, then primary and foreign keys are defined before loading the data into the SQL datastore.

B. Retrieval Flow

In a monolithic retrieval workflow, every query is typically handled the same way by expanding keywords and hitting a single database before returning the result. This one-size-fits-all approach causes issues such as blurring the user's intent, blind dispatch to a single datastore and inconsistent relevance. As such, the proposed architecture splits the retrieval flow into discrete stages to avoid the drawbacks of monolithic workflows while maximising retrieval accuracy and efficiency.

1. **Query Rewriting LLM**
   User inputs are transformed into concise, contextually rich prompts to align the queries with the indexing schemes of the datastores.
2. **Retriever Router Agent**
   The retriever router agent serves as the decision-making hub by analysing incoming queries and selecting the most appropriate datastore from the three options. This is done by employing semantic similarity routing to select the corresponding retriever for document retrieval, avoiding the one-size-fits-all limitation.
3. **Re-Ranker**
   Having retrieved the documents from the retrieval modes, the re-ranker merges and orders the documents by employing ranking algorithms to produce a relevance-ordered top-K list.
4. **Generation**
   The LLM combines the top results from the retrieved list of documents with the rewritten query and system instructions to generate a response to the user's query.

VI. Implementation Roadmap

Building upon the theoretical framework established in this review, the next steps will involve the following:

- **Document Readers & Parsers**
  - Develop parsers for PDF, Word, JSON, XML and CSV formats.
  - Build extraction pipelines to convert raw data into expected representations.

- **Datastore Ingestion**
  - Vector Datastore: Implement chunking process to segment text, integrate embedding model to vectorise each chunk.
  - Graph Datastore: Define graph schema, build entity and relation extraction model to populate graph nodes and edges.
  - SQL Datastore: Define normalised relational schemas, design ETL pipeline to load data into SQL datastore.
- **Query Routing Mechanisms**
  - Develop LLM for query rewriting/expansion to produce context-rich prompts.
  - Perform routing accuracy test on benchmark sets.
- **Security & Authentication**
  - Build database authentication that supports RBAC and ABAC enforcement.
  - Implement audit-trail logging across datastores.

VII. Conclusion

This literature review has examined the evolution of RAG systems, tracing their development from the foundational Naive RAG framework to the emerging Agentic RAG. Through thorough analysis of existing research, several key limitations have been identified in current single-store RAG approaches when confronted with real-world data complexity. Modern Organizations handle diverse data formats, mainly unstructured documents, structured tabular data and knowledge graphs with each requiring their own specific processing and retrieval methods. Current RAG systems rely mostly on vector datastores for semantic similarity search, creating a "one-size-fits-all" approach that struggles with complex relationship queries and structured data operations. Additionally, this analysis have reviewed that graph datastores enable multi-hop reasoning but lack efficient full-text search capabilities and SQL datastores provide precise filtering and aggregation but unable to handle unstructured content effectively.

Based on these findings, this review proposes an Agentic Multi-Store RAG (AMS-RAG) architecture that addresses the identified limitations through intelligent routing of queries to the specific datastores. The proposed architecture integrates vector datastores for semantic similarity search, graph datastores for relationship traversal and multi-hop reasoning, and SQL datastores for precise transactional queries. The dynamic classification of queries to the appropriate datastores is done by a central Retriever Router Agent and a re-ranking framework is implemented to merge results into a relevance ordered list before feeding to the LLM for response generation.

Additionally, this review identified a gap in existing RAG research regarding security and authentication frameworks for multi-store systems. The proposed AMS-RAG architecture addresses this oversight by utilising a database authentication system that provides dynamic authorization by incorporating centralised identity management and layered access control that combines RBAC and ABAC mechanisms. Furthermore, for auditing purposes, logging and monitoring features will be implemented across the datastores.

References

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Apr. 2021. Available: https://arxiv.org/pdf/2005.11401

[2] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," Mar. 2024. Available: https://arxiv.org/pdf/2312.10997

[3] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models Chain-of-Thought Prompting," Jan. 2023. Available: https://arxiv.org/pdf/2201.11903

[4] L. Gao, X. Ma, J. Lin, and J. Callan, "Precise Zero-Shot Dense Retrieval without Relevance Labels," Dec. 2022. Available: https://arxiv.org/pdf/2212.10496

[5] A. Singh, A. Ehtesham, S. Kumar, and Khoei, Tala Talaei, "Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG," arXiv (Cornell University), Jan. 2025, doi: https://doi.org/10.48550/arxiv.2501.09136.